

Dynamic Adjustment of Monte Carlo Tree Search Simulations for a 9×9 -Go-AI

Contents

1	Introduction	2
1.1	Gap	2
1.2	Hypothesis	2
2	Background knowledge	2
2.1	Go	2
3	Technical Foundations	3
3.1	Reinforcement Learning and Self-Play FILL ML nec?	3
3.2	Deep Neural Networks	3
3.2.1	Artificial Neural Networks	3
3.2.2	Residual Neural Networks	4
3.3	Monte Carlo Tree Search	4
4	Previous work	6
4.1	Pre-deep Learning	6
4.1.1	Before MCTS	6
4.1.2	After MCTS	6
4.2	Deep Learning	6
4.2.1	AlphaGo	6
4.2.2	AlphaZero	8
4.2.3	KataGo	8
5	Methodology	8
5.1	Reasoning 9×9 vs. 19×19	8
5.2	Reasoning AlphaZero vs. KataGo	8
5.3	Environment (Training server, implementation)	9
5.4	Heuristics	9
5.4.1	Difference of Probability for Likeliest K Actions	9
5.4.2	Difference of Highest and Lowest K Actions	9
5.4.3	Learning Complexity	9
5.4.4	Random	9
5.5	Evaluation	9
6	Experiments & Results	9
6.1	Using Heuristics	9
6.2	Using Heuristics in Play	9
7	Discussion	9
8	Conclusion	9
	Bibliography	10

1 Introduction

Creating a computer capable of playing games has been part of Computer Science research for a long time. It is a benchmark of human intelligence and an exploration of the boundaries of what is capable using artificial decision making. This culminated in the defeat of reigning world-chess champion, Garry Kasparov, at the computations of the IBM chess computer DeepBlue in 1997. [CITE HERE](#) This spurred on further improvement on chess machines and research on a multitude of other games. Another popular game by the name of Go remained a problem to scientists for 19 more years. While many approaches were tested, it was not until 2016 that a high-ranking professional player was beaten by Google-DeepMind's artificial intelligence (AI), AlphaGo, that used a vastly different approach from earlier game computers and programs. It combined Deep Neural Network (DNN) training with exploration guided by distributions output by the neural network. [CITE HERE](#)

1.1 Gap

A problem that is apparent with most AIs is the amount of computation that is necessary to achieve good results. The literature is abundant with examples of large networks and decision making algorithms that reasonably require too much computing power. Therefore, it is necessary to improve efficiency of algorithms by all possible means. Especially, algorithms that make use Monte Carlo Tree Search (MCTS) seem to tend to execute a lot of unnecessary computation because it executes the allowed number of simulations without consideration as to whether the return move probability distribution changes and such computation is necessary.

1.2 Hypothesis

The central claim of this thesis is that efficiency of MCTS can be increased by dynamically adjusting the amount of simulations based on the prior move probabilities generated by the DNN while upholding parity in performance. The development and implementation of such a heuristic are the goal of this thesis.

2 Background knowledge

2.1 Go

Go is a Chinese board game that can be traced back to 4,000 years ago. It is played by two-players using black and white stones. While usually played on a 19×19 -grid, other board sizes like 9×9 or 13×13 exist. The goal of the game is to enclose more territory, i.e., grid intersections, than the opponent.

The players place stones in turn; the player with the black stones begins. The group of free places around a single stone are called liberties, see Figure 1. Stones of the same colour that are placed on neighbouring fields are considered a group and share the liberties. A group may be arbitrarily large. A group of the opponent can be captured by filling in all liberties of the group, see Figure 2. While generally any stone may be placed anywhere on the board, it may not be placed in places where the stone or its group dies because of the stone being placed. However, such a stone may be placed if placing this stone means capturing an opponent's group. Importantly, a board position may not be immediately repeated. This means, after a stone has been captured, and the capturing stone may not be recaptured on the next move. This is called Ko. [FILL Appendix](#)

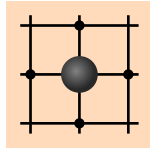


Figure 1: Liberties of stones

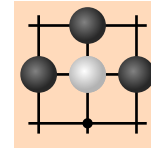


Figure 2: Capturing a Stone

The game ends when both players pass consecutively. Usually, players pass when all necessary moves are played and the territories are firmly established. Afterwards, the territory is counted. For that, players place back their prisoner stones into the opponent's territory. The remaining free grid intersections within their territory are the points for the respective player. To account for black's advantage in going first and avoid draw, additional 5.5-7.5 points are awarded to white. This is called komi.

Komi can be adjusted to even out the differences in capability between differently strong players. Lastly, komi can be combined with a handicap that places black stones on predetermined spots on the board.

Traditionally, players, human or computational, are attributed ranks based on strength. The weakest players play at 30 kyu. You can improve until reaching 1 kyu. From there, you can improve to 1 dan and theoretically up to 9 dan. Most top-level amateur players become 6 dan. Professional players are ranked separately with their own dan-scale. Nowadays, players are also evaluated using the Elo-system.

3 Technical Foundations

3.1 Reinforcement Learning and Self-Play **FILL ML nec?**

Reinforcement Learning (RL) describes a Machine Learning paradigm in which an agent is trained to maximize a reward signal in interaction with its environment. This makes use of two concepts called exploitation and exploration. To exploit is to choose a path of action that is known to produce a good result. On the other side, to explore is to choose a path of action that is not well-known and thus, upon further examination, may produce a better result than already known paths of action. Self-play is a specialised framework within the field of RL. In self-play, an algorithm plays a match in a game against an earlier iteration of itself. In this case, the previous iteration is the environment and the reward signal is the outcome of the match. Various selection strategies exist for the comparison-iteration, e.g., a random iteration is chosen to play, or the selection follows a "King-of-the-Hill"-approach, where the strongest iteration remains as the benchmark and is to be beaten. This, more than the randomised selection shows growth in performance of the agent.

3.2 Deep Neural Networks

3.2.1 Artificial Neural Networks

Today, artificial neural networks (ANN) are one of the most used and researched technologies. With their groundwork laid in 1958, new methods have been since been developed for a large number of purposes, not the least of which is playing games and improving beyond human capabilities.

Generally, ANNs receive an input of defined but arbitrary format. The signal of this input are propagated through the network to the output layer. One layer is made up of multiple neurons, each with their own weights per input, plus a bias individual to each neuron. In

the context to RL, the ANN's output is evaluated for its reward signal in response to the environment. From this, an error signal is generated and passed backwards through the network. Each layer is updated dependent on the succeeding layer's error signal.

3.2.2 Residual Neural Networks

In the course of research and development of ANNs, they have grown exponentially deeper. Beginning with a single perceptron, which were then used in a single layers, now, there are networks using neurons with more than 100 layers and a large number of perceptrons per layer. Users of ANNs of this size noticed the "Vanishing Gradient" problem. In the process of backpropagation, the derivatives' chain rule is employed. For that, multiple small decimal numbers are repeatedly multiplied. This produces even smaller decimal numbers. The "Vanishing Gradient" describes the phenomenon that the error signal that early layers of the networks receive is too small to effectively change the weights and biases of those layers. This stops learning from having any effect.

Hello

Figure 3: Illustration of Residual Neural Network

To solve this problem, residual neural networks were introduced in Deep Learning (DL) problems. Residual neural networks (ResNet) make use of "skipping connections." In one of such connections, a residual layer projects its output to another layer further down the stack of layers, skipping the layers inbetween. The residual output is added to the output of the layer, to which it is projected. This largely fixes the Vanishing Gradient since the output of the layer is passed further into the stack of layers.

A negative point in this setup is that the intermediate layers may be found to be superfluous and converge to 0 during training.

3.3 Monte Carlo Tree Search

MCTS is a heuristic decision making algorithm. Through exploration of subsequent states from a root state, MCTS finds a state that is most likely to maximise a metric that can be changed per use-case. During the search, a tree is built from the original state as root node R. There are four stages to MCTS:

1. Selection: From a parent node P, choose a child node according to a selection method. Repeat until choosing a leaf node C.
2. Expansion: If C is not a terminal state, i.e., there is a possible following state, create new child nodes. Choose a child state.
3. Evaluation: A state may be evaluated by choosing random subsequent states until a terminal state which has a definitive value. Another method is an evaluation function applied to the state.
4. Backpropagation: Update the probabilities in the path from C to R using the result from the rollout.

Through rigorous exploration of the tree by simulations, i.e., paths from root to terminal state, the child states most likely to maximise the evaluation method converge to higher values in the selection method.

The selection method most commonly used in the selection phase is the "Upper Confidence Bound 1 applied to Trees" (UCT). It is used to find a balance between exploitation of well-simulated moves and exploration of less-simulated moves in MCTS.

In the selection stage, the node with the highest value according to the expression $\frac{w_i}{n_i} + c\sqrt{\ln \frac{N_i}{n_i}}$ is selected, where

- w_i is the the wins for prospective C after the i -th move,
- n_i is the visits for prospective C after the i -th node,
- N_i is the visits for P after the i -th move, and
- c is the exploration parameter.